Exercise 1: Random Number Generator

Francesco Miranda Mendes*

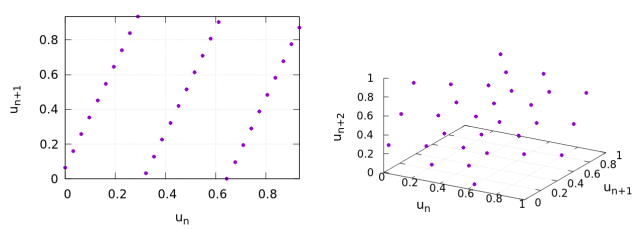
This work focused on the implementation of pseudo-random number generators such as the Linear Congruential Generator (LCG) and `rand()`, as well as on the analysis of their quality through the square, cube and χ^2 tests. Using the LCG, random points were generated within a circle.

I. IMPLEMENTATION OF A LINEAR CONGRUENTIAL GENERATOR

We began by implementing a linear congruential generator, which generates X_{n+1} pseudo-randomly according to the equation

$$X_{n+1} = (cX_n + a) \; mod \; p \tag{1}$$

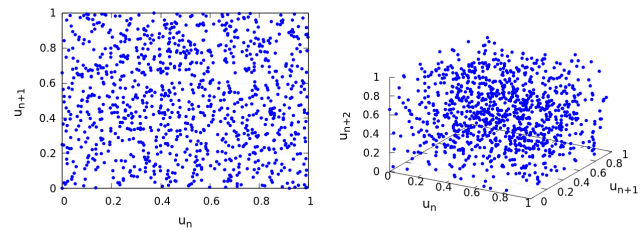
where c, a and $mod\ p$ represent, respectively, the multiplier, the increment, and the integer division by the period. A good pseudo-random generator is characterised by producing **uncorrelated** numbers that follow a **uniform distribution**. Taking $X_0 = 5$ as the seed of the algorithm and $N = 1000$ as the maximum number of random numbers generated, we began by setting $c_1 = 3$, $a = 2$ and $p_1 = 31$. In order to assess the quality of the generator and, more specifically, the dispersion and correlation of the generated points, the square test and the cube test were performed:



(a) Square test (2D) (b) Cube test (3D)

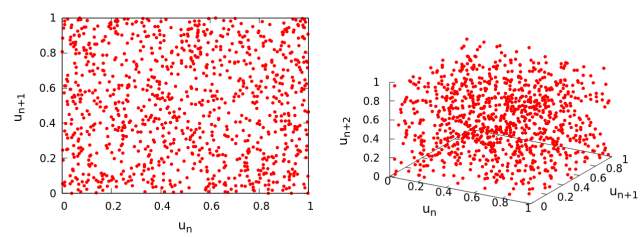
Figure 1: Generation of 1000 pseudo-random numbers belonging to $[0;1]$ ($u_n = X_n/p$) using the LCG with *bad parameters*.

As can be seen in Figure 1, the generated numbers create a well-defined pattern of mutually parallel lines, from which it can be stated that these numbers are linearly correlated. Furthermore, it is noted that, for $N = 1000$ generated points, only 30 are observed, which is consistent with a consequence of the definition of the period p, which states that the maximum number of distinct numbers that can be generated is given by $N_{max} = p - 1$. Thus, the remaining points are superimposed. From this fact, it can be seen that the generated numbers are far from following a uniform distribution (poorly dispersed across the plot). We can therefore conclude that the LCG, for these parameters, is a *bad generator*.



(a) Square test (2D) (b) Cube test (3D)

Figure 2: Generation of 1000 pseudo-random numbers using the LCG with $c_2 = 16807$, $a = 2$ and $p_2 = 2^{31} - 1 = 2147483647$.



(a) Square test (2D) (b) Cube test (3D)

Figure 3: Generation of 1001 random numbers using the `rand()` function from $C++$, with $x_0 = 7$ as the seed.

Comparing Figure 1 and Figure 2, it is easy to see that the LCG, for the parameters of the 2nd case, is a better generator than the LCG with the first parameters, since no patterns are observed (there is no correlation between points), and because the points are fairly dispersed across the plot, which allows us to state that they approximate a uniform distribution. Both observations are also verified in Figure 2, for `rand()`. It is worth noting that for 1000 generated numbers, 1000 distinct values were obtained in the case of this LCG, and `rand()`, for 1001 generated numbers, produced 1001 distinct values, which is a good indicator. Therefore, the LCG for these parameters, and `rand()`, are good random number generators. With only these two tests, it is not possible to compare the LCG with good parameters against `rand()`.

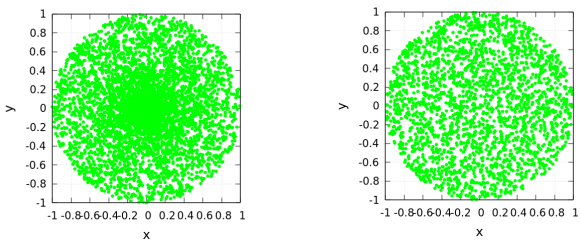
* fc59899@alunos.fc.ul.pt

II. GENERATING POINTS UNIFORMLY WITHIN A CIRCLE

In this section we were asked to generate points within a circle without rejection. The starting point is the polar coordinate system:

$$\begin{cases} x = rcos\theta \\ y = rsin\theta \end{cases} \tag{2}$$

Restricting r to $[0; R]$, with $R = 1$, and θ to $[0; 2\pi]$, we proceed to generate r and θ randomly using the linear congruential generator, obtaining a circle of radius R, as follows



(a) $r = r$ (b) $r = \sqrt{U}$

Figure 4: Generation of $N = 5000$ random points without rejection. As seeds of the LCG for r and θ, $x_{0r} = 145$ and $x_{0\theta} = 698$ were chosen, respectively. The LCG parameters that optimise it (2nd case from the previous section) were used. After x_r and x_θ were generated, they were divided by p to belong to $[0; 1]$. Furthermore, after dividing, x_θ was multiplied by 2π to fall within $[0; 2\pi]$.

Looking at Figure 4, it is immediately apparent that there is a large concentration of points at the centre of the circle compared to the remaining area, which goes against the objective of creating points that follow a uniform distribution. It was seen in the previous section, through the correlation tests, that the points generated by the LCG, for these parameters, are a good approximation of a uniform distribution, so the quality of the generator is not in question. In truth, what compromises the generated distribution is the fact that the area of the circle grows with r^2 ($A = \pi r^2$), so the density of points per unit area is no longer constant. To correct this bias, the Inverse Transform Method was used, which allows random variables with a given distribution to be generated from a uniform variable. In a circle, the probability of a point falling between radii r and $r + dr$ is proportional to the area of the corresponding annulus $dP = 2\pi r dr$. Normalising to the total area gives

$$p(r)dr = \frac{2\pi r dr}{\pi R^2} = \frac{2r}{R^2} \tag{3}$$

which represents the Probability Density Function. The cumulative function $F(r)$ is obtained by integrating the density:

$$F(r) = \int_0^r p(r')dr' = \frac{r^2}{R^2} \tag{4}$$

Defining a uniform random variable U on $[0; 1]$ such that $U = F(r)$, it follows that $r = R\sqrt{U} = \sqrt{U}$. The final result is a circle with points approximately uniformly distributed, as can be seen in Figure 4 (b).

III. THE χ^2 TEST

In this section the conclusions drawn earlier regarding the quality of the pseudo-random generators will be confirmed through the χ^2 test, which is defined as follows:

$$\chi^2 = \sum_{i=1}^{k} \frac{(N_i - np_i)^2}{np_i} \tag{5}$$

where k is the number of intervals, N_i is the observed count in each interval i, n is the total number of observations, and p_i is the expected probability for interval i (in this case, $p_i = \frac{1}{k}$ since we are implementing ideally uniform distributions). It should be noted that if the distribution is perfectly uniform then $N_i - np_i = 0 \Rightarrow \chi^2 = 0$. The following results are obtained for $k = 51$:

Generator	χ^2	**p-value (%)**	**n (points)**
Linear Congruential (p_1,c_1)	700.034	≈ 100	1000
Linear Congruential (p_2,c_2)	54.476	[50, 75]	1000
rand()	49.172	[50, 75]	1000
drand48()	54.476	[50, 75]	1000

Table I: Values of χ^2 and p for the different generators, for $\nu = k - 1 = 50$ degrees of freedom.

Analysing Table 1, it is concluded that the statements made in Section 1 are confirmed. Indeed, for the 1st set of parameters, the LCG leaves much to be desired, since it presents a very high χ^2 and a p-value of approximately 1. The p-value indicates the probability that the obtained result is compatible with a uniform distribution. Intermediate values of p (around 50%) mean that the generator behaves in a statistically consistent manner with randomness. The LCG for the 2nd set of parameters and `rand()` perform quite well, as both have a low χ^2 on the order of ν. It can also be stated that `rand()` is a slightly better generator, since $\chi^2_{rand()}$ is closer to ν than χ^2_{LCG}. The test was also applied to `drand48()`, yielding a χ^2 value identical to that of the LCG (2nd case), from which it can be said that they are generators of very similar quality.